



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G01

SYSTEMS INTEGRATION

AND ARCHITECTURE 1

SOFTWARE REQUIREMENTS

SPECIFICATION (SRS)

Project Title: Unisell: A Centralized Multi-Platform Ecommerce Management System

Prepared By: Prince Jason Capendit

Date of Submission: 07/01/26

Version: 2.0

Revision History Table

Version	Date	Description
1.0	June 23, 2026	Initial SRS submitted. Provide project background, problem statement, general bullet and narrative paragraph description of high level project objectives, scope, limitations, overall description, operating environment, assumptions and dependencies, six system features. There is no requirement ID, no traceability table, no business rules section, no stakeholders section. UML diagrams were annotated with "To Be Finalized".
2.0	July 1, 2026	Completed SRS after the Requirements Engineering process. The following changes have been done over Version 1.0: Restructured the whole document, taking the proposal style sections and replacing them with Introduction, Stakeholders and Users, Functional Requirements, Non-Functional Requirements, Business Rules, Requirements Traceability Table and UML Diagrams. Added Stakeholders and Users section with Guest Users, Vendors, Administrators, Developer and Course Instructor and their roles. Replaced the previous narrative paragraph and general bullet format with a set of numbered and testable functional requirements (FR-001 - FR-032). Added three new functional requirements (FR-007, FR-008, FR-009) for per-platform Stock Allocation, prevent over-allocating stock and display per-platform Stock Breakdown which corresponds with the addition of per-platform Stock Management as a core feature of V2.0. Introduced FR-010 which requires vendors to specify the origin platform when creating an order, and introduces platform-specific stock allocation for order creation. Mapped all non-functional requirements, and presented them as discrete numbered requirements (NFR-001 to NFR-027), grouped into six categories: Performance, Security, Usability, Reliability, Compatibility, and Maintainability. Set out of the box, unspecified NFR values: session timeout (NFR-009, 30 minutes), failed login attempts (NFR-011, 3), and uptime (NFR-017, weekly). Added Business Rules section (BR-001 to BR-013) for system-level policies that include those for account uniqueness, per-platform stock constraints, return record generation, vendor data isolation, payment status progression, order status progression, chat message retention, session expiry and manual vs. automatic stock

		adjustment boundaries. Added Platform entity and Inventory junction entity to the ERD and connected Product and Platform with per-platform stock allocation. Newly added ReturnRecord and ChatMessage entities, that were missing from the ERD altogether. Entities that are not part of the scope of the Unisell business model were removed from the ERD: Cart, CartItem, Review, Discount and Customer as a separate entity. Instead, customer data (name and address) was directly embedded as attributes of the Order entity. The one-to-order or order-to-product cardinalities have been corrected for Order-to-Payment to be one-to-one, Order-to-Shipment to be one-to-one and Order-to-ReturnRecord to be one-to-zero-or-one, and the direct Order-to-Payment relationship has been deleted and replaced with the correct Order-to-PaymentProduct relationship. Include Additional Requirements Traceability Table – FR-001, ..., FR-032, associated use cases, and UML models. Filled in five UML diagrams that were previously marked as "To Be Finalized": Use Case Diagram, Entity-Relationship Diagram, Activity Diagrams (User Login, Add Order, Process Return), and Sequence Diagrams (User Login, Add Order, Process Return). Explicitly added per-platform stock allocation as a V2.0 feature to the Scope of the System. Combined the original Limitations section into a renamed Assumptions and Constraints section for consistency in structure with finalized format. Lastly, added an Individual Work declaration at the end of the document.
--	--	--

Table of Contents

1. Introduction	5
1.1. Project Title	5
1.2. System Overview	5
1.3. Purpose of the System	5
1.4. Target Users	5
1.5. Problem Statement	5
1.6. Scope of the System	5
1.7 Assumptions and Constraints	6
2. Stakeholder and Users	6
3. Functional Requirements	7
4. Non-Functional Requirements	10
5. Business Rules	11
6. Requirements Traceability Table	12
7. UML Diagrams	15
7.1 Use Case Diagram	15
7.2 Entity - Relationship Diagram	16
7.3 Activity Diagram	17
7.4 Sequence Diagram	20
8. Initial Project Timeline	23
9. References	23
10. Appendices	24

1. Introduction

1.1. Project Title

Unisell: A Centralized Multi-Platform Ecommerce Management System

1.2. System Overview

Unisell is a cross-platform ecommerce management tool that can be accessed on both a web app and an Android app, so vendors can get a hold of product management, stock tracking, orders, payments and shipping for all products in one place rather than using different dashboards on marketplaces like Shopee, Lazada or TikTok Shop.

1.3. Purpose of the System

Unisell aims to eliminate the inaccuracies, overselling, and time loss caused by vendors manually managing their business's stock and orders across multiple platforms by integrating inventory, order, payment, and shipment management into one centralized system.

1.4. Target Users

Independent online vendors who sell across multiple e-commerce platforms, and system administrators who oversee vendor activity and platform health.

1.5. Problem Statement

Handling stock manually across various platforms is prone to inaccuracies, overselling, and time loss for vendors. A centralized system is needed to consolidate order, payment, and shipment management.

1.6. Scope of the System

Unisell is a cross-platform e-commerce management system accessible through a web application and an Android mobile application. The system covers the following main features:

- Vendor and admin authentication with role-based access control.
- Product and category management, including adding, editing, and removing listings.
- Real-time inventory updates are defined as stock changes reflected within 2 seconds of a triggering action, such as a placed order or approved return.
- Per-platform stock allocation, allowing vendors to assign a specific portion of a product's total stock to each platform (e.g., Shopee, Lazada, TikTok Shop), preventing overselling on any single platform while maintaining one shared inventory pool.

- Order management supports multiple items per order, with full status tracking from placement to delivery.
- Payment tracking per order, viewable by both vendor and admin.
- Shipment tracking with manual update capability by the vendor.
- Automatic return record generation for orders marked as uncollected or returned.
- A vendor-to-admin chat feature for direct in-system communication.
- Reporting tools that summarize orders, inventory levels, and payments per vendor.
- Guest users are limited strictly to the home, login, and registration pages.

1.7. Assumptions and Constraints

Assumptions

Version 2.0 assumes and relies on the following:

- It is assumed that a vendor has a stable Internet connection to access and use the platform.
- The system is powered by Render, MySQL, and Vercel or Netlify for its backend, database, and frontend, respectively. If any of these services are down, the availability of the system will be impacted.
- In this version, there is no automated data import, so the inventory needs to be manually encoded by the vendors.
- Version 2.0 runs with one currency and locale.
- It is assumed that all users will be using a modern and up-to-date web browser (Chrome, Firefox, Edge, or Safari).

Constraints

Version 2.0 does not include the following:

- Direct access to platforms like Shopee, Lazada, or TikTok Shop. This will be in a future version.
- Predictive inventory forecasting or advanced analytics.
- The real-time integration with third-party logistics providers.
- Video or voice conversations in vendor-managed chat.
- Data import tools for bulk importing existing inventory. During the initial process of onboarding, vendors will be required to enter all of their products by hand.

2. Stakeholders and Users

Stakeholder/User	Role in the System
Guest User	A visitor who can only access the home,

	login, and registration pages before becoming a registered vendor.
Vendor	A business owner who handles products, stock, orders, payments and shipping on their own and is able to chat with administrators.
Administrator	A system manager with an overview of all vendor accounts, monitoring, vendor concern resolution through chat, and report generation of system-wide reports.
Developer	Responsible for designing, building, testing, and maintaining the system (Prince Jason Capendit).
Course Instructor	Indirect stakeholder; evaluates the system as an academic requirement for IT342 (Mr. Frederick L. Revilleza Jr).

3. Functional Requirements

3.1. Centralized Inventory Management

Vendors can manage their product listings and stock levels from a single dashboard. Inventory is updated automatically when orders are placed, or returns are processed, and vendors are notified when stock runs low.

- **FR-001:** The system shall enable the vendors to add a new product by entering the product name, SKU, price, quantity available, the product category, and save it to their inventory.
- **FR-002:** The system shall allow the vendors to have the ability to edit an existing product's information in their inventory, such as name, price, quantity, and category, and save the updated information.
- **FR-003:** The system shall allow vendors to remove a product from their inventory, making it no longer visible or accessible within the system.
- **FR-004:** The system shall automatically reduce the allocated stock quantity in Inventory for the specific platform on which the order was placed.
- **FR-005:** The system shall automatically restore the available stock quantity of a product in a vendor's inventory when a return request for that product is approved.
- **FR-006:** The system shall notify the vendor through a visible alert on their dashboard when the stock level of any product falls below a set minimum threshold, defaulting to 5 units if no custom threshold has been configured.

- **FR-007:** The system shall allow vendors to allocate a specific stock quantity to a product for each platform, without exceeding the product's total available stock.
- **FR-008:** The system shall prevent a vendor from saving platform stock allocations that, combined, exceed the product's total stock quantity.
- **FR-009:** The system shall display each product's stock breakdown per platform on the vendor's dashboard.
- **FR-010:** The system shall require vendors to select the platform an order originated from when adding a new order, and shall associate the order with that platform's allocated stock.

3.2. Order and Shipment Management

Vendors can view and process customer orders from the moment they are placed until they are either delivered or returned. The system handles return records automatically for orders that are uncollected or sent back.

- **FR-011:** The system shall provide the vendors with a list of all orders placed in their account, along with order ID, items, quantity, total payment, and the order status.
- **FR-012:** The system shall offer the vendor options to indicate when an order has been placed, is being processed, or has been shipped, delivered, or returned, including Pending, Processing, Shipped, Delivered, and Returned options.
- **FR-013:** The system will enable vendors to update the shipment tracking number and courier information for an order that is set to be Shipped.
- **FR-014:** The system should allow for a return record to be automatically generated when the order is marked as Returned or when the Shipment is marked as uncollected, with all details of the return, including Order ID, affected items, Return reason, and Date of return.
- **FR-015:** The system shall allow administrators to be able to see all return records for all vendor accounts, including vendor name, order ID, items returned and the date that the return was recorded.
- **FR-016:** The system shall provide the vendor with the status of the payment made on each order which implies whether the payment has been received, pending or refunded after an order returned.

3.3. User Authentication and Role-Based Access Control

The system allows safe login and registration for all users. The users' access to features and pages is tightly guarded in accordance with their role, so vendors, administrators, and guests only view what they have access to.

- **FR-017:** The system shall allow guest users to register for a vendor account by providing a name, email address, and password.
- **FR-018:** The system shall allow registered vendors and administrators to log in using their email address and password.
- **FR-019:** The system shall restrict guest users to only the home, login, and registration pages, preventing access to any vendor or admin features.
- **FR-020:** The system shall automatically redirect vendors to their dashboard upon successful login and administrators to their own separate dashboard.
- **FR-021:** The system shall log out a user and invalidate their session after a period of inactivity.
- **FR-022:** The system shall prevent vendors from accessing any data, orders, or products that belong to another vendor account.

3.4. Payment Tracking

Vendors can monitor the payment status of every order placed under their account. Administrators can also view payment records across all vendors for oversight purposes.

- **FR-023:** The system shall allow vendors to view the payment status of each order, displayed as Pending, Received, or Refunded.
- **FR-024:** The system shall automatically update the payment status of an order to Refunded when a return record for that order has been approved.
- **FR-025:** The system shall allow administrators to view payment records across all vendor accounts, including the vendor name, order ID, amount, and current payment status.

3.5. Vendor-to-Admin Chat

Through an in-system chat, vendors can initiate discussion directly with administrators. This enables vendors to voice concerns, ask questions and seek support without exiting the site.

- **FR-026:** The system shall allow vendors to send a text message to an administrator through the in-system chat, with the message delivered and displayed within 1 second.
- **FR-027:** The system shall allow administrators to view and respond to messages from any vendor through a centralized chat interface.
- **FR-028:** The system shall display all previous messages in a conversation in chronological order whenever a vendor or administrator opens the chat.
- **FR-029:** The system shall persist all chat messages so that no message is lost in the event of a page refresh or session timeout.

3.6. Reporting and System Oversight

Administrators have access to system-wide reports and vendor monitoring tools to ensure smooth platform operations and informed decision-making.

- **FR-030:** The system shall allow administrators to generate a report summarizing the total number of orders, inventory levels, and payment statuses per vendor.
- **FR-031:** The system shall allow administrators to view and manage all vendor accounts, including the ability to activate or deactivate an account.
- **FR-032:** The system shall allow administrators to view a log of system activity, including login events, order status changes, and return record generation.

4. Non-Functional Requirements

4.1. Performance

- **NFR-001:** Normal user interactions (for example, page navigation, form submission) shall respond within 2 seconds under normal operating conditions.
- **NFR-002:** The system shall be able to serve at least 100 concurrent users without a significant performance drop.
- **NFR-003:** Any webpage should be loaded within 3 seconds under normal network conditions.
- **NFR-004:** Deliver and display chat messages between vendor(s) and administrator(s) within 1 second of message being sent.

4.2. Security

- **NFR-005:** All data sent between the client applications and the backend server should be encrypted with HTTPS.
- **NFR-006:** All user passwords will be stored in the database after being hashed with BCrypt.
- **NFR-007:** The system shall implement role-based access control to enable restrictions on features and data that can be accessed by guest users, vendors, and administrators based on their role.
- **NFR-008:** All data entered in the system by the user should be checked for validity and tampering on the client side and server side to ensure that invalid or tampered data is not processed.
- **NFR-009:** If a user is inactive for 30 minutes, the system shall end the user's session.
- **NFR-010:** A critical actions log should be kept for the system that records login attempts, order status changes, and account changes.
- **NFR-011:** The system should not allow more than three failed logon attempts in a day to avoid brute-force attacks.

4.3. Usability

- **NFR-012:** The system should have a smooth, intuitive interface that does not need extensive training to use effectively, for new vendors and administrators.

- **NFR-013:** The system shall display clear and descriptive error messages whenever a user action cannot be completed.
- **NFR-014:** The system shall provide search and filtering capabilities for vendors browsing their products and orders.
- **NFR-015:** The system shall present a responsive layout that adapts to desktop, tablet, and Android mobile screen sizes.
- **NFR-016:** The system shall maintain distinct and clearly differentiated interfaces for vendor and administrator users.

4.4. Reliability

- **NFR-017:** The system shall maintain a minimum of 99% uptime, measured on a weekly basis.
- **NFR-018:** The system shall perform automated daily backups of all stored data, including products, orders, payments, and chat messages.
- **NFR-019:** The data consistency will be maintained across the web application and Android application at all times, on the shared backend and shared database.
- **NFR-020:** The system should recover from errors gracefully without data loss, and provide a user-friendly message to the user.
- **NFR-021:** The system should remember everything it has said and nothing will be lost if a page is refreshed, the app is restarted or the session expires.

4.5. Compatibility

- **NFR-022:** The system's web application shall ensure that the web application of the system works properly on the latest version of Chromium, Firefox, Edge, and Safari browsers.
- **NFR-023:** The Android app for the system will work on Android devices that have at least 2Gb of RAM.
- **NFR-024:** The system should have consistent functionality and data synchronization between the Web application and the Android application at all times.

4.6. Maintainability

- **NFR-025:** The system must be designed and constructed in a modular architectural style, such that there is the ability to make changes in one component of the system without affecting features of another unrelated component.
- **NFR-026:** The system shall have a documented code base that can be reviewed and understood by new developers, and minimal time spent to get up to speed with the project.

- **NFR-027:** The system shall ensure that vendor data is separated at the Database level so that the data from one vendor does not impact or be accessed by another vendor.

5. Business Rules

- A vendor account must be associated with a unique, valid email address. Also, the system shall not allow two accounts to register using the same email.
- An order shall not be confirmed if the requested quantity of a product exceeds that product's allocated stock quantity for the specific platform on which the order was placed, even if the product has additional unallocated stock available on other platforms or in its total stock pool.
- The sum of a product's stock allocated across all platforms shall not exceed its total available stock.
- A return record must be automatically generated whenever an order's status is changed to returned or when a shipment is marked as uncollected. This cannot be skipped or manually bypassed by the vendor.
- A vendor shall only be able to view, edit, or manage their own products, orders, payments, and shipments, and shall never have access to another vendor's data.
- Only administrators are authorized to activate or deactivate a vendor account.
- An order's payment status may only move from pending to received, or from received to refunded, following an approved return. It cannot move backward from refunded to received.
- An order's status must progressively be one of the following: pending, processing, shipped, delivered, or returned. It cannot skip directly from pending to delivered without passing through processing and shipping.
- All communications made by a vendor and an administrator will be kept and not deleted or removed by either party.
- Guest users cannot access any part of the inventory, order, payment or chat features until they have registered and been approved as a vendor.
- A user's session will expire after 30 minutes of inactivity. Afterwards, it now requires the user to log in again to continue using the system.
- A product's inventory value can be updated manually by the vendor to update the stock to reflect restocking or a physical inventory count.
- The system shall operate under a single currency and locale for all transactions in Version 2.0.

6. Requirements Traceability Table

Requirement ID	Requirement Description	Related Use Case	UML Model(s)
----------------	-------------------------	------------------	--------------

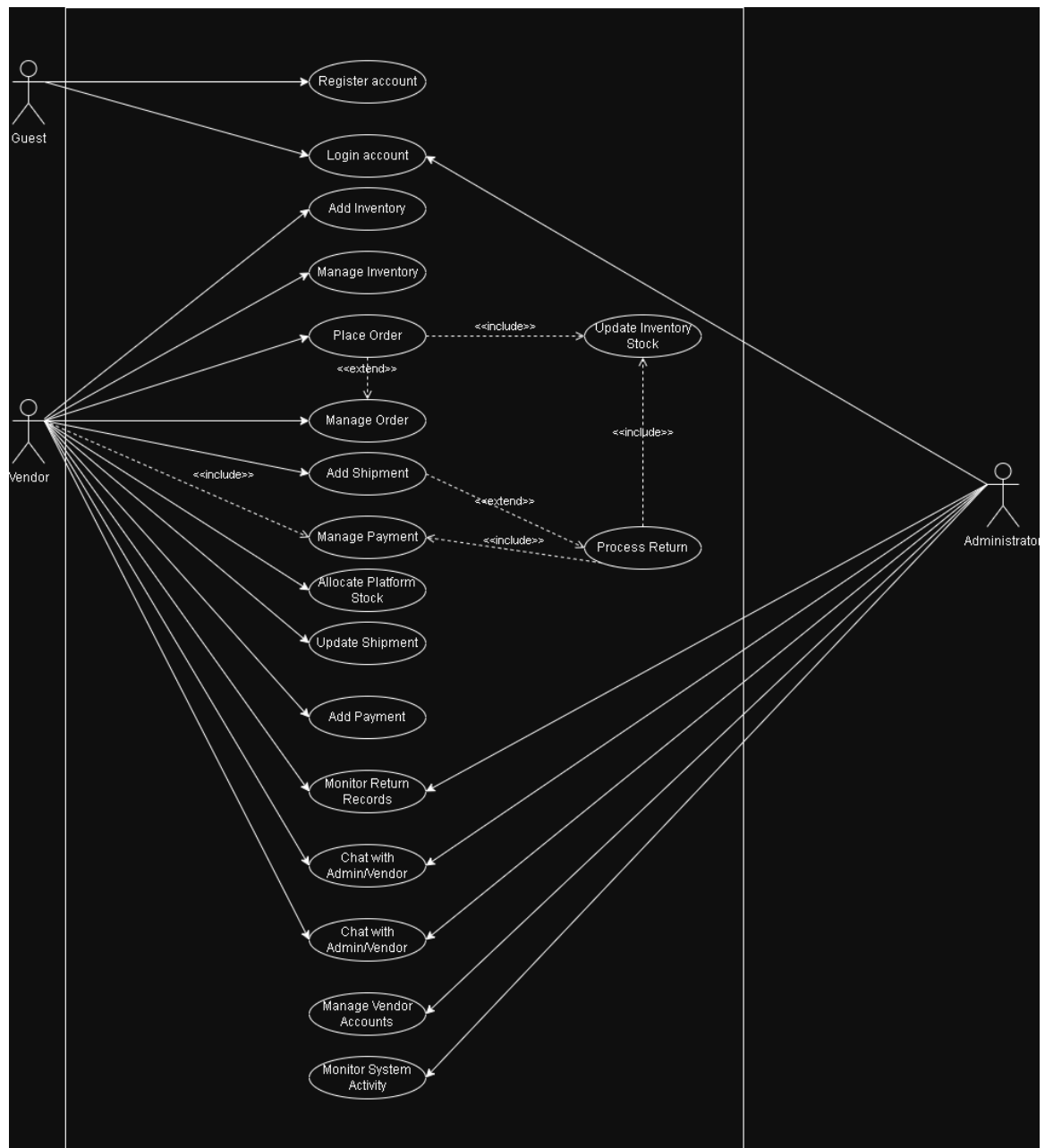
FR-001	Add new product	Manage Inventory	Use Case Diagram, ERD, Class Diagram
FR-002	Edit existing product	Manage Inventory	Use Case Diagram, ERD, Class Diagram
FR-003	Remove product	Manage Inventory	Use Case Diagram, ERD
FR-004	Auto-reduce stock on order confirmation	Place Order	Use Case Diagram, Activity Diagram, Sequence Diagram
FR-005	Auto-restore stock on approved return	Process Return	Use Case Diagram, Activity Diagram, Sequence Diagram
FR-006	Low-stock alert notification	Manage Product	Use Case Diagram, Activity Diagram
FR-007	Allocate platform-specific stock	Allocate Platform Stock	Use Case Diagram, Sequence Diagram
FR-008	Prevent over-allocation beyond the total stock	Allocate Platform Stock	Use Case Diagram, Activity Diagram
FR-009	Display per-platform stock breakdown	Manage Inventory	Use Case Diagram, ERD
FR-010	Select the platform when adding an order	Add Order	Use Case Diagram, ERD, Sequence Diagram
FR-011	View order list	Manage Order	Use Case Diagram, Sequence Diagram
FR-012	Update order status	Manage Order	Use Case Diagram, Activity Diagram, Sequence Diagram
FR-013	Update shipment tracking details	Manage Shipment	Use Case Diagram, Sequence Diagram
FR-014	Auto-generate return record	Process Return	Use Case Diagram, Activity Diagram, ERD, Sequence Diagram
FR-015	Admin view of all return	Monitor Returns	Use Case Diagram,

	records		ERD
FR-016	View order payment status	Manage Order	Use Case Diagram, ERD
FR-017	Vendor registration	User Registration	Use Case Diagram, Sequence Diagram
FR-018	User login	User Login	Use Case Diagram, Sequence Diagram
FR-019	Restrict guest access	User Login	Use Case Diagram
FR-020	Role-based dashboard redirect	User Login	Use Case Diagram, Activity Diagram
FR-021	Session timeout/logout	User Login	Use Case Diagram, Activity Diagram
FR-022	Vendor data isolation	Manage Inventory / Manage Order	Class Diagram, ERD
FR-023	View payment status	Manage Payment	Use Case Diagram, ERD
FR-024	Auto-update payment status on return	Process Return	Use Case Diagram, Activity Diagram, Sequence Diagram
FR-025	Admin view of payment records	Monitor Payments	Use Case Diagram, ERD
FR-026	Send chat message	Vendor-Admin Chat	Use Case Diagram, Sequence Diagram
FR-027	Admin respond to chat	Vendor-Admin Chat	Use Case Diagram, Sequence Diagram
FR-028	View chat history	Vendor-Admin Chat	Use Case Diagram, ERD
FR-029	Persist chat messages	Vendor-Admin Chat	ERD, Sequence Diagram
FR-030	Generate vendor summary report	Generate Report	Use Case Diagram, Sequence Diagram
FR-031	Manage vendor accounts (activate/deactivate)	Manage Vendor Accounts	Use Case Diagram, Activity Diagram
FR-032	View system activity log	Monitor System	Use Case Diagram,

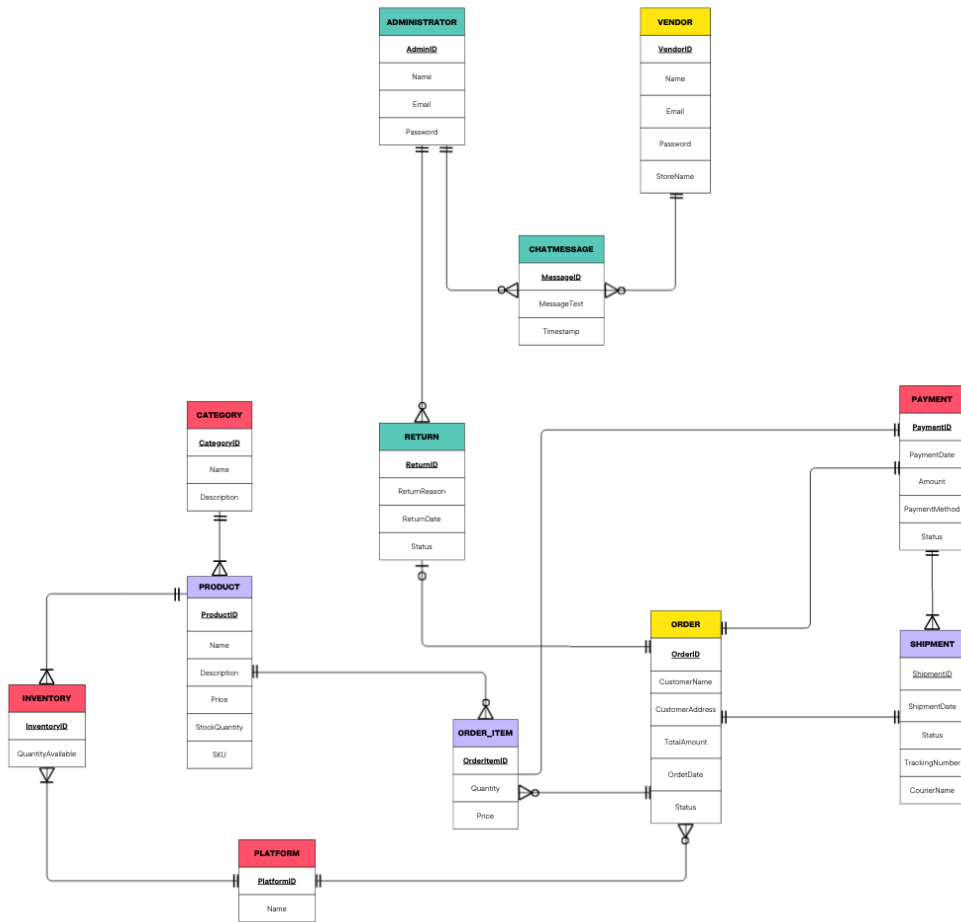
		Activity	ERD
--	--	----------	-----

7. UML Diagrams

7.1. Use Case Diagram

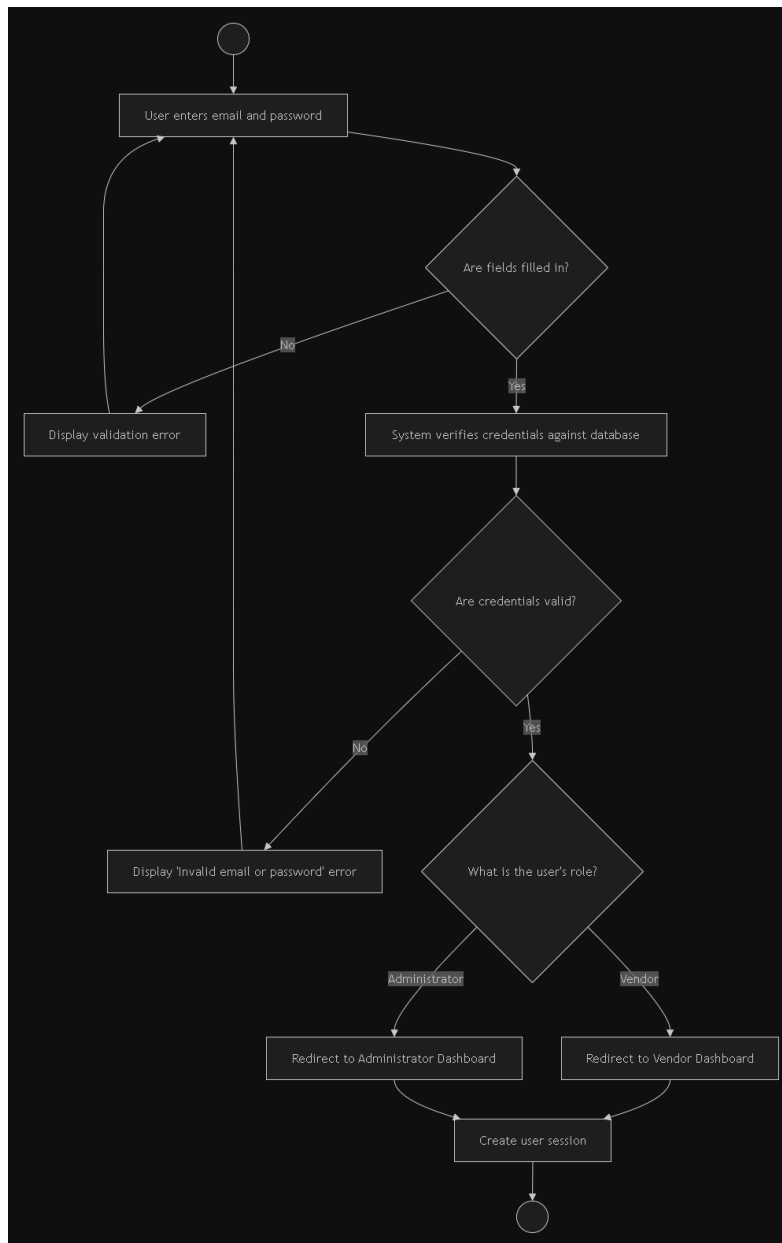


7.2. Entity - Relationship Diagram

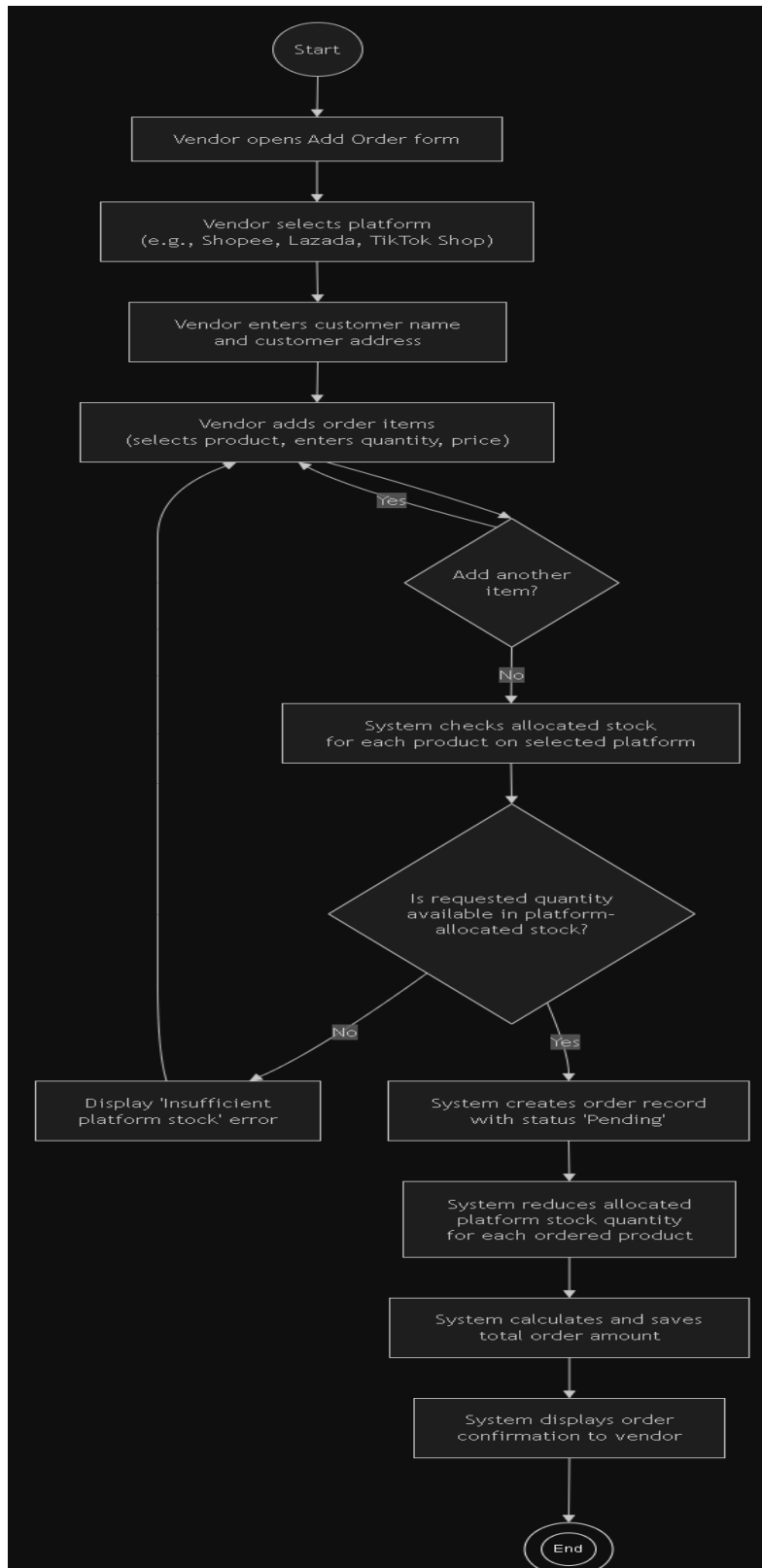


7.3. Activity Diagram

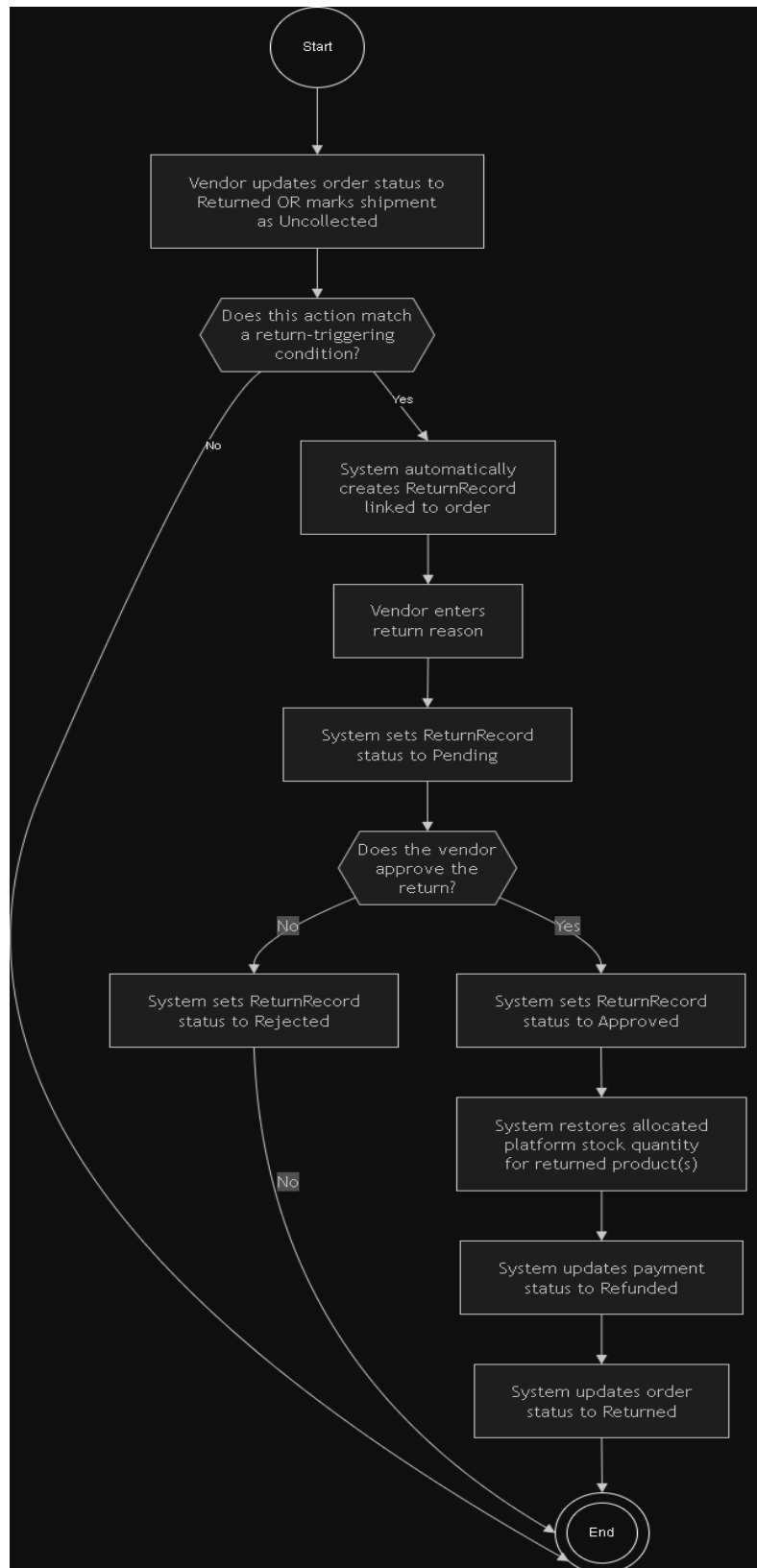
User Login



Add Order

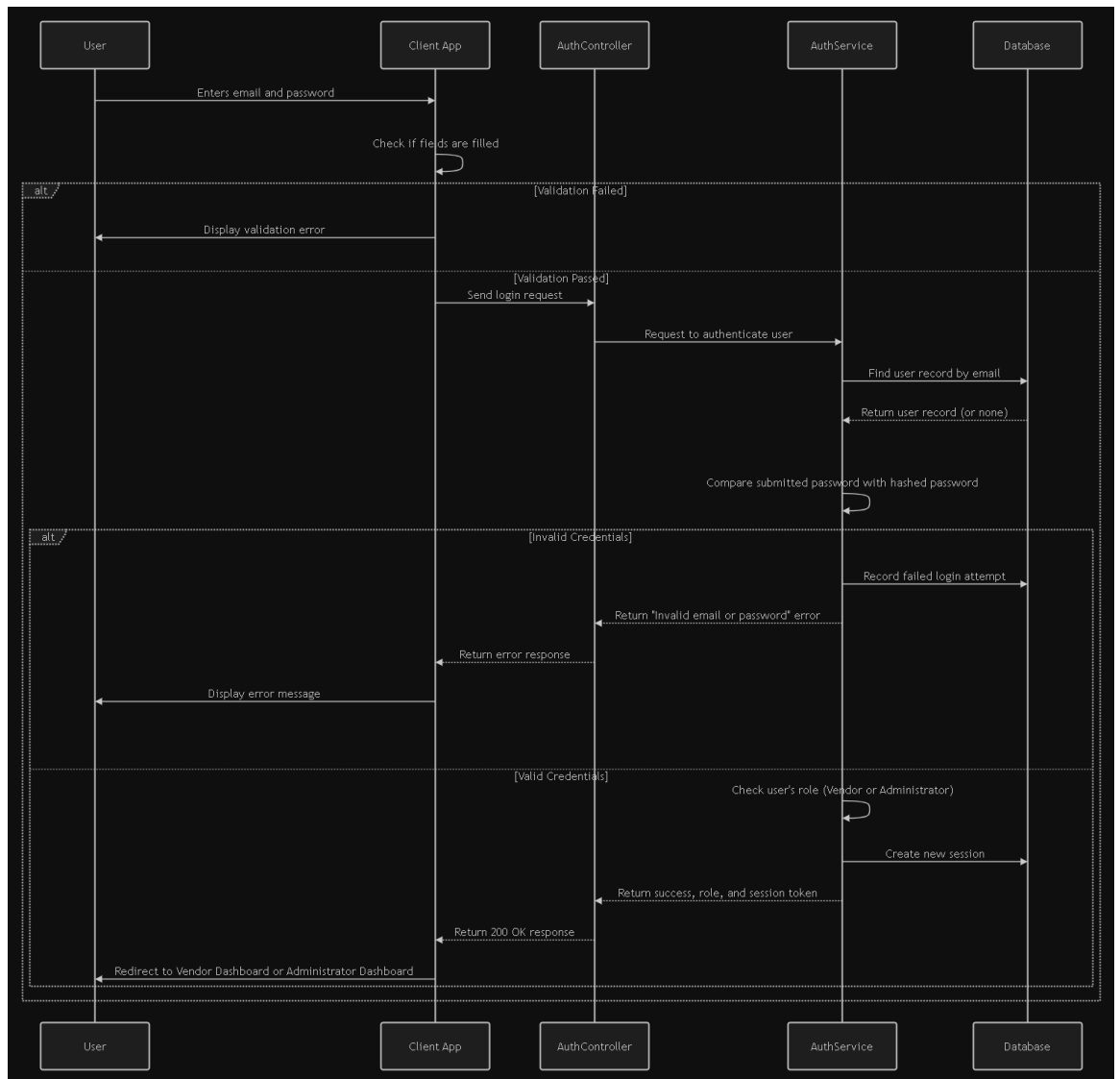


Process Return

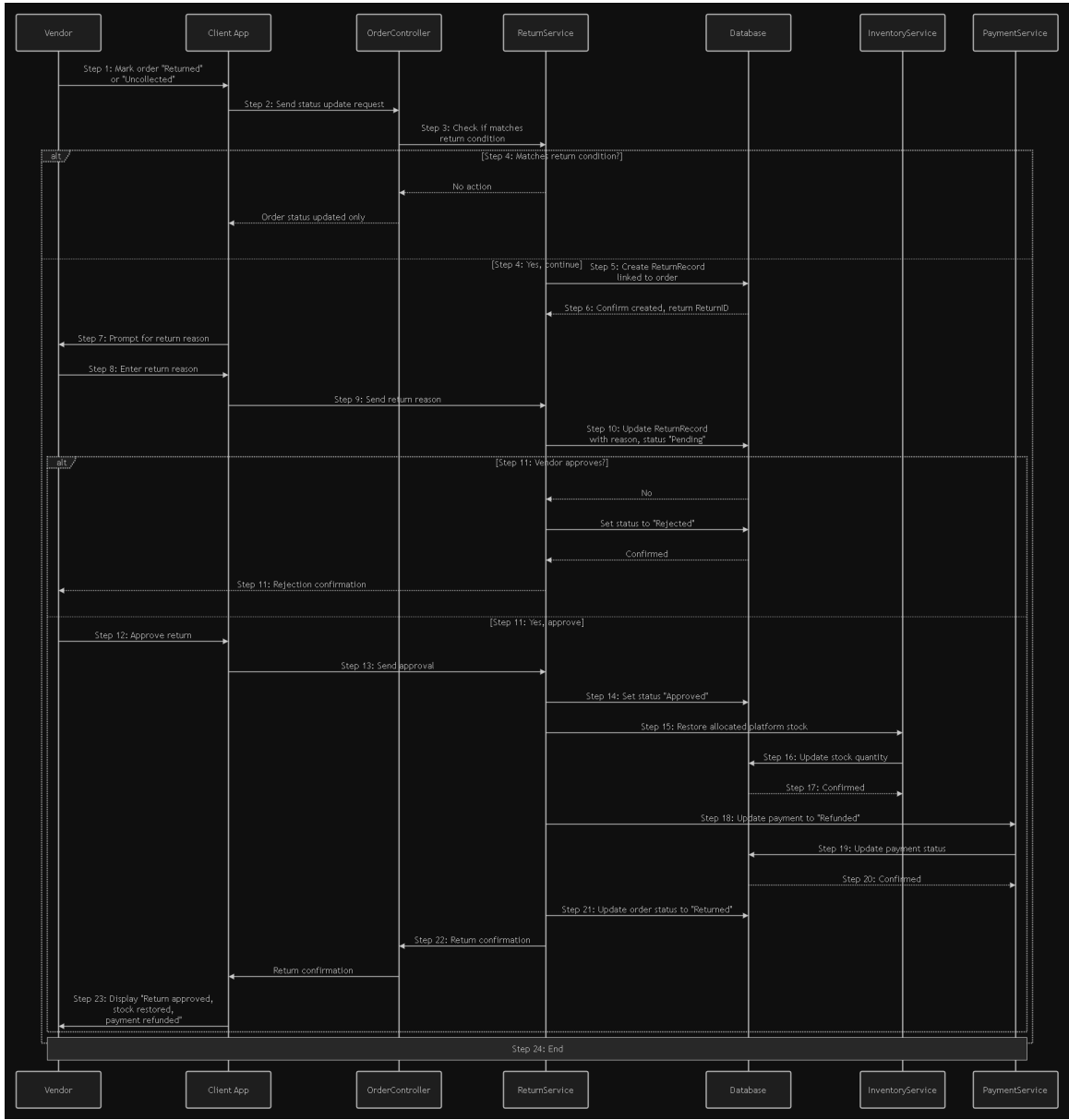


7.4. Sequence Diagram

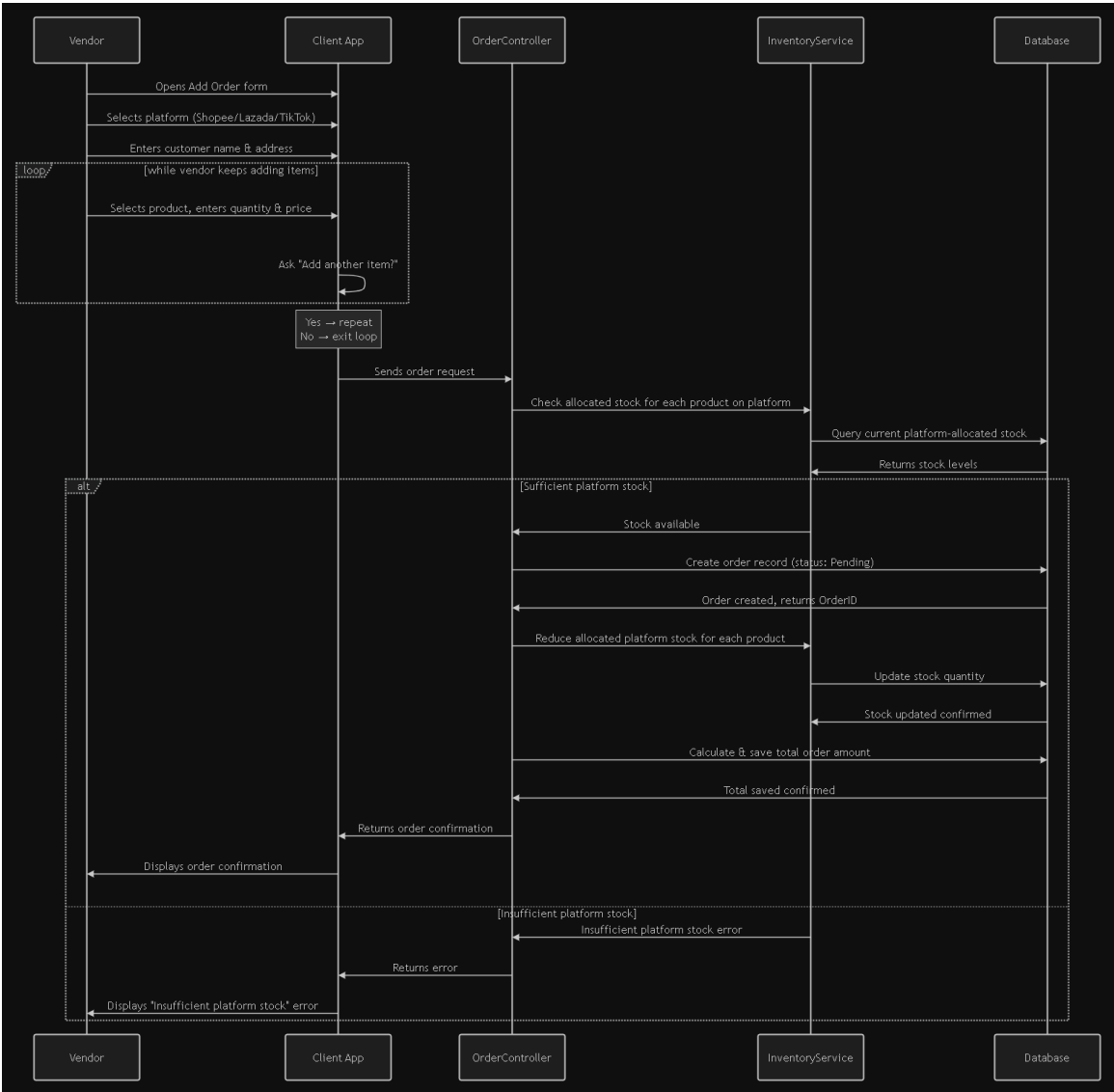
User Login



Process Return



Add Order



8. Initial Project Timeline

Week	Planned Activity
Week 1	Proposal preparation and approval, requirements refinement, database schema design, UI/UX wireframes, system architecture planning
Week 2	Spring Boot backend setup, MySQL database integration, authentication module development, core API endpoint development
Week 3	ReactJS web application development, Kotlin Android application development, API integration, inventory and order management module implementation
Week 4	System integration testing, bug fixing, use acceptance testing, deployment preparation, final documentation update, SRS revision

9. References

Spring Boot. (n.d.). Spring Boot Reference Documentation. VMware Tanzu.

<https://spring.io/projects/spring-boot>

React. (n.d.). React Documentation. Meta Open Source.

<https://react.dev>

Android Developers. (n.d.). Kotlin for Android. Google.

<https://developer.android.com/kotlin>

Oracle. (n.d.). MySQL Documentation.

<https://dev.mysql.com/doc>

Postman. (n.d.). Postman Learning Center.

<https://learning.postman.com>

IEEE. (1998). IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. IEEE Standards Association.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

Fielding, R. T. (2000). Architectural Styles and the Design of Network-Based Software Architectures. University of California, Irvine.

10. Appendices

Appendix A: Glossary

Term	Definition
Vendor	Any registered user who deals with products, inventory, order, payments and shipping using the Unisell platform.
Administrator	A user with elevated privileges responsible for managing vendor accounts, monitoring system activity, generating reports, and providing support.
Product	An item listed by a vendor for sale and managed within the system inventory.
Inventory	A record that tracks the stock quantity of a product allocated to a specific platform, which is linked to both the product and the platform within the system.
Order	A transaction record containing one or more purchased products and their associated details.
Payment Status	The current state of payment for an order, such as Pending, Received, or Refunded.
Platform	A third party e-commerce platform where the seller assigns the stock of a product in the Unisell system.

Shipment	The tracking data on an order, such as the tracking number, carrier, and delivery status.
Return Record	A record automatically generated when an order is returned or marked as uncollected.
Chat Message	A text-based communication exchanged between vendors and administrators through the integrated messaging feature.
REST API	A collection of endpoints that are used for communication between the front-end applications and the back-end server.
RBAC	RBAC is a security system that controls access to the system based on the roles of its users.
SRS	Software Requirements Specification, document that specifies the requirements for the system, including functional and non-functional requirements.
ReactJS	A JavaScript library used for developing the web application frontend of Unisell.
Spring Boot	A Java-based framework used to develop the backend API and business logic of Unisell.
Kotlin	A programming language used to develop the Android mobile application of Unisell.
MySQL	A relational database management system used to store and manage all system data.

Appendix B: Technology Stack Summary

Component	Technology
Frontend (Web)	ReactJS

Frontend (Mobile)	Kotlin (Android)
Backend	Spring Boot
Database	MySQL
API Architecture	RESTful API
Authentication	Role-Based Access Control (RBAC) with BCrypt Password Hashing
Development Tools	IntelliJ IDEA, Android Studio, Visual Studio Code, MySQL Workbench, Postman

System Architecture Overview:

The architecture of the Unisell system consists of a web client application done in ReactJS, a mobile client app done in Kotlin, and a centralized Spring Boot backend that communicates with the clients via RESTful APIs. The back-end will handle business logic, and all data in the system will be stored in a MySQL database. The distributed architecture provides uniformity of data synchronization across all platforms, scalability, maintainability, and security.

Individual Work

I certify that this finalized SRS and UML models are my own individual work. I understand that copied, duplicated, or AI-generated submissions without proper understanding and revision may be subject to verification and possible deductions.